

Post-Quantum Secure and Dynamic Load-Balanced Encryption for IIoT Data in Fog Computing

Phinnawat Yaemsanguan, Rangsimann Sattayarom,

Ratchanon Wongwitutai, Napha Mongkolwittayakul, Somchart Fugkeaw

Sirindhorn International Institute of Technology, Thammasat University, Thailand

6622770764@g.siiit.tu.ac.th, 6622771747@g.siiit.tu.ac.th, 6622780268@g.siiit.tu.ac.th, 6622782330@g.siiit.tu.ac.th

Abstract—Industrial Internet of Things (IIoT) systems generate high-volume data streams that increase device-to-cloud latency and impose heavy pressure on network and cloud infrastructures. Fog computing addresses this challenge by moving computation and data processing closer to the network edge. However, conventional secure fog-assisted IIoT frameworks often focus mainly on encryption while overlooking the computational overhead introduced by post-quantum cryptography and the heterogeneous resource constraints of fog nodes. These challenges become more significant in practical deployments, where fog nodes differ in CPU capability, memory capacity, and network conditions, causing static load-sharing mechanisms to perform poorly under bursty traffic.

In this work, a post-quantum fog-assisted security architecture for IIoT is proposed. The system integrates PUF-based device authentication, Kyber-based post-quantum key establishment during the setup phase, and ChaCha20-Poly1305 for lightweight authenticated protection of IIoT data streams. Multiple protected data streams generated by IIoT devices are collected and aggregated at the edge layer, where a group-based authentication mechanism is applied to reduce repeated authentication overhead before forwarding the validated batch to the fog layer. To support fine-grained access control under resource-constrained fog environments, a split-phase lattice-based CP-ABE mechanism is adopted, in which sensitive cryptographic operations are executed within Trusted Execution Environments (TEEs) while complementary processing is handled outside the enclave. After secure aggregation, the fog layer re-encrypts the protected batch using AES for subsequent storage and controlled access. In addition, a dynamic load-balancing mechanism coordinated by a Master Fog Node assigns security-related tasks to heterogeneous fog nodes according to real-time resource availability. The proposed design aims to improve secure data handling efficiency while maintaining post-quantum protection and scalable fog-layer processing for IIoT environments.

Index Terms—Industrial Internet of Things (IIoT), Fog Computing, Post-Quantum Cryptography (PQC), Lattice-Based CP-ABE, Dynamic Load Balancing, Trusted Execution Environment (TEE), Physical Unclonable Function (PUF), Fine-Grained Access Control.

I. INTRODUCTION

Industrial Internet of Things (IIoT) environments consist of large numbers of sensors, machines, and controllers that continuously generate data at high volume and velocity. As industrial systems scale, the amount of streaming data increases significantly, placing heavy pressure on communication networks and centralized cloud infrastructures. Transmitting raw IIoT data directly to the cloud often leads to bandwidth congestion, increased latency, and inefficient use of network

and computing resources. Fog computing addresses these limitations by relocating computation and storage resources closer to IIoT devices, thereby reducing communication delay and network load.

However, ensuring secure and efficient data handling in fog-assisted IIoT environments remains a major challenge. Many existing approaches focus primarily on data encryption while giving limited attention to the computational overhead of cryptographic protection, especially in high-volume and real-time industrial settings. In practice, IIoT data streams must be authenticated, protected, aggregated, and forwarded across multiple layers under strict latency constraints. Treating all tasks uniformly can introduce unnecessary overhead and reduce system efficiency, particularly when heterogeneous processing resources are involved.

These challenges become more critical in the post-quantum era. Post-quantum cryptographic mechanisms generally incur higher computational and communication overhead than conventional public-key schemes, making their direct deployment difficult in large-scale IIoT systems. Moreover, improper placement of post-quantum operations across the IIoT–fog–cloud pipeline may increase end-to-end latency, reduce throughput, and overload resource-constrained components. Therefore, designing an efficient security workflow that determines how authentication, key establishment, encryption, and access control should be distributed across the edge, fog, and cloud layers becomes an important research problem.

The problem is further complicated by the heterogeneous nature of fog environments. Fog nodes often differ in processing capability, memory capacity, and network conditions, while Trusted Execution Environments (TEEs) provide limited secure memory for sensitive cryptographic operations. In practical deployments, each fog node can host multiple concurrent TEE instances (enclaves), enabling parallel execution of security-sensitive workloads. However, this multi-enclave capability introduces additional challenges in enclave memory management, workload isolation, and scheduling efficiency, particularly under dynamic and high-throughput IIoT conditions. As a result, improper workload placement may lead to enclave contention, EPC memory pressure, and performance degradation, making TEEs a potential throughput bottleneck despite their strong security guarantees. Consequently, efficient workload management must operate at multiple levels, including inter-node load balancing across heterogeneous fog

nodes and intra-node scheduling between TEE and REE execution environments. This necessitates an intelligent, enclave-aware scheduling mechanism capable of jointly optimizing performance, security, and resource utilization under dynamic system conditions. In many current deployments, workload distribution is still handled using static load balancing or fixed routing strategies, such as threshold-based offloading or pre-determined forwarding rules. These approaches cannot adapt effectively to bursty traffic, changing network conditions, or diverse fog-node capabilities. As a result, computationally intensive security tasks may be assigned to unsuitable fog nodes, creating bottlenecks, queue buildup, and service degradation.

To address these challenges, this work proposes a post-quantum fog-assisted security architecture for IIoT environments. The proposed system integrates PUF-based device authentication, Kyber-based post-quantum key establishment during the setup phase, and ChaCha20-Poly1305 for lightweight authenticated protection of IIoT data streams. Multiple protected streams are collected and aggregated at the edge layer, where a group-based authentication mechanism is applied to reduce repeated verification overhead before forwarding validated batches to the fog layer. To support secure storage and fine-grained access control, the fog layer re-encrypts aggregated protected data using AES and applies a split-phase lattice-based CP-ABE mechanism, in which sensitive cryptographic operations are executed within Trusted Execution Environments while complementary processing is handled outside the enclave. In addition, a dynamic load-balancing mechanism coordinated by a Master Fog Node assigns security-related workloads according to real-time resource availability. Through this design, the proposed system aims to improve secure data handling efficiency while maintaining low-latency processing and post-quantum resilience in heterogeneous fog-assisted IIoT environments.

In summary, the main contributions of this work are as follows:

- 1) **End-to-End Post-Quantum Secure IIoT Architecture** We design a unified security architecture spanning device, gateway, and fog layers. At the device side, a PUF-based hardware root of trust is combined with CRYSTALS-Kyber to derive a session key for authenticated encryption via ChaCha20-Poly1305. At the fog layer, sensitive operations are protected within TEEs, where secure decryption and aggregation are performed, followed by a split-phase lattice-based CP-ABE encryption that confines key-dependent computations inside enclaves while offloading policy expansion to the REE.
- 2) **Secure and Efficient Micro-Batch Ingestion** We introduce a gateway-side validation and micro-batch formation mechanism that filters malicious packets using lightweight authentication and constructs integrity-protected batches with deterministic aggregation, enabling scalable and reliable ingestion of high-throughput IIoT data streams.
- 3) **Capability-Aware Fog Orchestration** We propose a dynamic Master Fog Node (MFN) selection mechanism

based on a capability-aware readiness metric that captures real-time workload, enclave availability, and cryptographic processing capacity. This enables robust coordination of security-sensitive workloads in heterogeneous fog environments.

4) **Spider: Hierarchical Multi-Enclave Load Balancing**

We propose **Spider++**, a novel two-level scheduling framework that performs (i) inter-node load balancing across fog nodes and (ii) intra-node scheduling across multiple parallel TEE enclaves. By incorporating enclave-level metrics such as queue state and EPC constraints, Spider++ enables parallel secure processing, mitigates enclave contention, and significantly improves throughput compared to conventional load balancing schemes.

II. RELATED WORK

A. Fog-Assisted IIoT Security and TEE-Based Execution

Fog computing has emerged as a key paradigm for enabling low-latency processing and distributed security services in Industrial Internet of Things (IIoT) systems. By bringing computation closer to end devices, fog nodes reduce communication overhead and improve responsiveness. However, ensuring secure authentication, fine-grained access control, and protected execution in distributed environments remains challenging.

Lightweight authentication in fog-assisted IIoT is commonly achieved using Physical Unclonable Function (PUF)-based mechanisms. For instance, [1], [2] propose PUF-based authentication schemes that support group authentication and efficient device verification with reduced computational overhead. While these approaches improve scalability and eliminate heavy key storage, they primarily focus on authentication and message integrity, without addressing fine-grained multi-user data sharing and access control.

To enable secure data sharing, Ciphertext-Policy Attribute-Based Encryption (CP-ABE) has been widely adopted. Works such as [3], [5] provide fine-grained access control and revocation in fog-assisted cloud environments, while [4] further introduces a lattice-based quantum-resistant CP-ABE scheme for IoT systems. Although these solutions strengthen confidentiality and access control, they operate independently of lightweight authentication mechanisms and do not consider hardware-assisted secure execution.

To protect sensitive computation, Trusted Execution Environments (TEEs) have been investigated at the edge and fog layers. Paper [6] identifies SGX-related limitations, including performance overhead, restricted enclave memory, and I/O security issues, while [7] shows that practical TEE implementations remain vulnerable to software, architectural, side-channel, and microarchitectural attacks. More broadly, fog-security frameworks [8]–[10] address trust-based secure computing, lightweight authentication and key management, and risk-aware access control, but none of them jointly integrate lightweight authentication, post-quantum access control, and TEE-assisted secure processing into a unified framework.

TABLE I
COMPREHENSIVE FEATURE COMPARISON OF RELATED WORKS AND PQ-SPIDER

Scheme	PUF	PQC	Fine-Grained AC	Revocation	TEE Support	Fog/Edge	Load Balancing
[4]	×	✓	✓	✓	×	✓	×
[5]	×	×	✓	✓	×	✓	×
[22]	×	×	×	×	×	✓	✓
[34]	×	✓	×	×	✓	×	×
[35]	×	✓	×	×	×	×	✓
[36]	×	✓	×	×	×	×	×
[37]	×	×	×	×	×	✓	✓
[39]	×	×	×	×	×	✓	✓
[40]	×	×	×	×	×	✓	✓
PQ-SPIDER (Ours)	✓	✓	✓	✓	✓	✓	✓

Therefore, a gap remains for a unified fog-assisted IIoT framework that combines lightweight authentication, post-quantum fine-grained access control, and TEE-assisted secure processing.

B. Post-Quantum Cryptography for IIoT Systems

The rapid development of quantum computing threatens conventional public-key cryptographic schemes such as RSA and ECC, motivating the adoption of post-quantum cryptography (PQC). Among the major PQC families, lattice-based cryptography has gained significant attention because of its strong security foundation and computational efficiency.

CRYSTALS-Kyber and CRYSTALS-Dilithium have emerged as leading lattice-based candidates for key encapsulation and digital signatures, respectively. Prior studies [12], [13] demonstrate their practicality in IIoT and network-stack settings while also highlighting performance trade-offs. In particular, [12] integrates lattice-based ML-KEM and ML-DSA into TLS 1.3 and X.509 for IIoT systems, while [13] benchmarks Dilithium/Kyber and Falcon/Kyber stacks, showing trade-offs in execution cost, workload distribution, and bandwidth.

For resource-constrained environments, [14], [15] further examine the deployment of PQC in IoT and IIoT settings. In particular, [14] adapts CRYSTALS-Kyber to wireless and device-to-device communication for constrained devices, whereas [15] evaluates PQC algorithms in IIoT scenarios in terms of execution time and energy consumption, showing that most PQ KEMs are competitive with classical curves and that hybrid PQC-classical combinations introduce only limited additional overhead.

To strengthen hardware-level protection, [16] combines Kyber with PUF and TRNG mechanisms to reduce reliance on stored secrets and improve physical security. Complementary hardware-rooted works such as [17] also show that PUF-based authentication can strengthen resistance to impersonation and physical attacks in IoT settings, although they are not themselves full PQC deployment frameworks.

Broader studies such as [18], [19] discuss hybrid quantum-resistant architectures and PQC adoption in wider networked and edge-fog environments. These works emphasize issues such as crypto-agility, scalability, interoperability, and deploy-

ment complexity when integrating post-quantum protection into practical communication systems.

Despite these advances, existing studies mainly focus on standalone PQC deployment, protocol benchmarking, or authentication enhancement. The integration of Kyber/Dilithium-based protection into fog-assisted IIoT architectures, particularly in combination with hardware-bound authentication, fine-grained access control, and secure execution, still remains limited. This gap motivates the need for a more integrated fog-assisted PQC framework.

C. Load Balancing in Heterogeneous Fog Environments

Efficient load balancing is critical in fog-assisted IIoT systems because fog nodes are inherently heterogeneous in computational capability, latency, and energy profile. To improve system performance under such conditions, dynamic scheduling strategies have been widely adopted. Prior works such as [22] dynamically allocate workloads based on system conditions, while [24], [25] consider latency, energy consumption, and broader fog-cloud resource factors. Multi-objective optimization approaches such as [26] further attempt to balance performance metrics including delay, cost, and energy consumption.

More recent studies have explored the interaction between load balancing and security. For example, [27] incorporates energy-aware scheduling near industrial devices, while [28] introduces secure and energy-aware load balancing in an SDN-enabled fog environment. In addition, [29], [30] show that trusted execution environments such as TrustZone or TEE introduce non-negligible overhead. In particular, [29] shows that REE-TEE interaction and secure task execution increase runtime and energy cost, while [30] explicitly highlights cryptographic-operation overhead and the single-processor monopoly of TrustZone, both of which directly affect scheduling decisions.

Table I presents a feature-level comparison of existing approaches in fog-assisted IIoT environments. Prior works address important aspects such as fine-grained access control and revocation in [4], [5], and dynamic load balancing in fog systems in [22], [37], [39], [40]. In particular, the CP-ABE-based schemes in [4], [5] provide fine-grained access control and revocation capabilities, with [4] further supporting

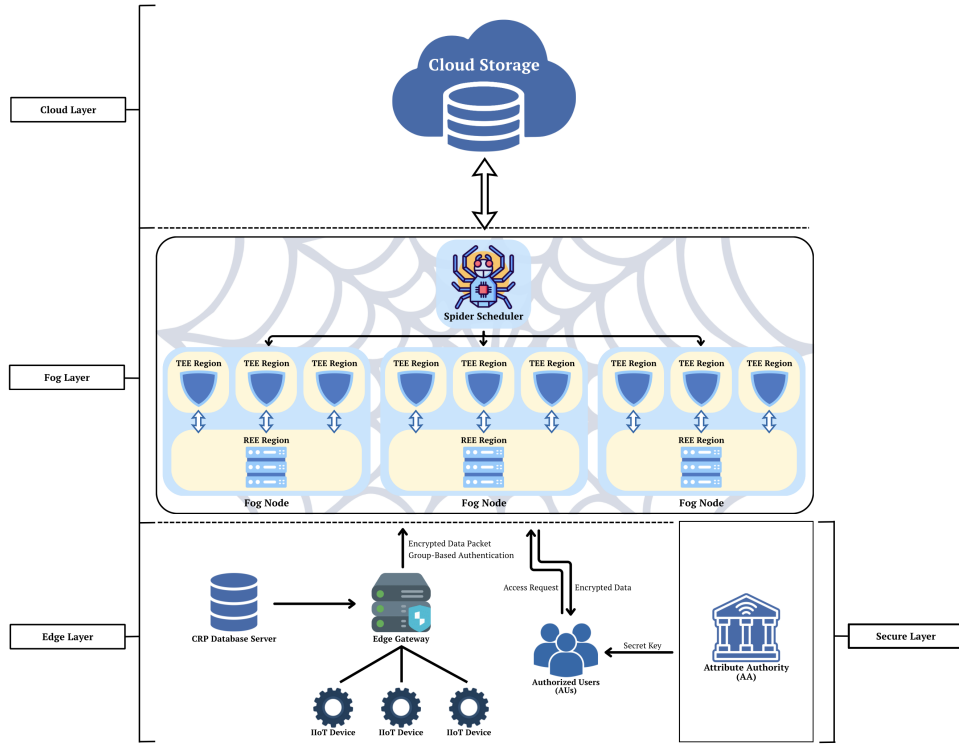


Fig. 1. Spider-Enabled Fog Computing Architecture for Secure IIoT Data Processing

a quantum-resistant construction. However, these security-focused schemes do not consider the interaction between access control mechanisms and runtime resource scheduling in fog-assisted settings. Conversely, existing fog-based load balancing approaches [22], [37], [39], [40] demonstrate the effectiveness of dynamic task allocation in improving system performance, but they generally focus on latency, energy consumption, or workload distribution under conventional security assumptions. As a result, they do not explicitly account for the additional computational overhead introduced by post-quantum cryptography or hardware-assisted secure execution environments.

Overall, existing works provide valuable foundations in authentication, access control, and load balancing, but they still leave open challenges in understanding how security mechanisms, particularly lightweight authentication, post-quantum protection, and TEE-assisted secure processing, interact with adaptive load balancing in heterogeneous fog environments. This gap motivates the need for a framework that jointly considers security overhead and runtime scheduling efficiency in fog-assisted IIoT systems.

III. OUR PROPOSED SYSTEM

This section presents the proposed post-quantum fog-assisted security architecture for the Industrial Internet of Things (IIoT), enhanced with a novel Spider-based intelligent load balancing mechanism.

A. System Model

As illustrated in Fig. 1, the proposed system adopts a layered architecture consisting of four layers: *Edge Layer*, *Fog Layer*, *Cloud Layer*, and *Secure Layer*. A key feature of the design is the integration of the **Spider Scheduler** within the fog layer, enabling security-aware, capability-aware, and *hierarchical enclave-aware* workload orchestration across distributed fog resources.

1) Layered Architecture Overview:

- **Edge Layer:** Data generation, device authentication, and initial aggregation.
- **Fog Layer:** Secure processing and Spider-based workload scheduling.
- **Cloud Layer:** Storage of encrypted IIoT data and meta-data.
- **Secure Layer:** Key management and access control via the Attribute Authority.

2) *System Entities and Spider Integration:* The system consists of the following components:

- 1) **Attribute Authority (AA)** initializes the lattice-based CP-ABE scheme, issues attribute keys, and defines access policies.
- 2) **IIoT Devices** are resource-constrained nodes equipped with PUFs for hardware-rooted identity. They perform lightweight authenticated encryption before transmission.
- 3) **Edge Gateway (GW)** authenticates devices via CRP validation, verifies packet integrity, and aggregates pack-

ets into batches

$$B_k = \{p_1, \dots, p_n\}$$

which are forwarded to the fog layer.

- 4) **CRP Database Server** stores challenge-response pairs for PUF-based authentication.
- 5) **Master Fog Node (MFN) with Spider Controller** orchestrates scheduling via the **Spider Scheduler**, which performs multi-factor routing based on queue state, EPC availability, latency, workload intensity, node capability, and trust condition. The engine includes batch profiling, state monitoring, score computation, node selection, and secure delegation.
- 6) **Fog Nodes (FN)** are TEE-enabled servers hosting multiple enclaves

$$F_j = \{E_{j,k}\}_{k=1}^{m_j}$$

Each enclave operates independently with local state (queue, service rate, EPC).

Fog nodes follow a **dual execution model**:

- **TEE**: Secure operations (key handling, aggregation, partial CP-ABE).
- **REE**: Non-sensitive tasks (policy expansion, formatting).

Spider performs **hierarchical scheduling**:

- Inter-node (across fog nodes),
- Intra-node (across enclaves).

A bounded resource model ensures stable operation under EPC constraints.

- 7) **Spider-Based Secure Delegation** supports workload migration using post-quantum encapsulation (e.g., Kyber) to protect intermediate states.
- 8) **Cloud Storage** is honest-but-curious and stores only encrypted data.
- 9) **Authorized Users (AUs)** decrypt data if their attributes satisfy CP-ABE policies.

B. Threat Model

We consider a strong PPT adversary with full control over public communication channels in the IIoT environment, targeting devices, networks, fog infrastructure, and cloud storage.

Adversary Capabilities. The adversary can observe, inject, modify, replay, and delay messages, and may compromise components outside trusted execution boundaries.

- **Device Attacks**: Impersonation, data forgery, and replay of captured transmissions.
- **Network Adversary**: Full control of communication links, enabling eavesdropping, modification, replay, delay, and DoS attacks.
- **Compromised Fog (REE)**: The REE may be fully compromised, allowing memory inspection and execution manipulation, while TEE preserves enclave confidentiality and integrity.
- **Faulty/Malicious Fog Nodes**: Nodes may produce incorrect results or delays due to faults or overload; Spider mitigates this via trust- and capability-aware scheduling.

TABLE II
CORE NOTATION USED IN SECTION III

Symbol	Description
λ	Security parameter.
PP, MSK	Public parameters and master secret key (CP-ABE).
SK_u	Secret key of user u .
$Attr_u$	Attribute set of user u .
ID_i	Identifier of device i .
B_k	Micro-batch generated at gateway.
$\mathcal{P}_{valid}$	Set of validated packets.
CT_i, Tag_i	Authenticated ciphertext and tag.
K_{AES}	Symmetric encryption key.
CT_{L-ABE}	Final CP-ABE ciphertext.
Q_j^{TEE}, Q_j^{REE}	TEE/REE queue lengths at node F_j .
μ_j^{TEE}, μ_j^{REE}	TEE/REE service rates.
$T_{wait}(F_j)$	Estimated waiting time.
$\rho_{epc}(F_j)$	EPC memory penalty.
$\mathcal{A}$	Access policy.
σ	Post-quantum signature.
Ω	Final data uploaded to cloud.

- **Scheduler Attacks**: Injection of false node metrics or workload manipulation to disrupt scheduling decisions.
- **TEE Side-Channel Attacks**: Leakage via timing or memory access patterns; partially mitigated but not eliminated.
- **Unauthorized Access**: Adversaries attempt to bypass CP-ABE policies to access encrypted cloud data.
- **Data Integrity Attacks**: Tampering with ciphertext, metadata, or intermediate results.
- **Quantum Adversary**: Capable of breaking classical cryptography; security relies on post-quantum assumptions.

C. System Process

The proposed scheme operates through seven phases: system initialization, hardware-bound IIoT encryption, gateway validation and micro-batch formation, capability-aware leader election, dynamic load balancing, TEE-assisted split-phase CP-ABE processing, and authorized user decryption.

Phase I: System Initialization: The Attribute Authority (AA) initializes the lattice-based CP-ABE system by generating public parameters and master secret keys. To preserve semantic privacy against untrusted host environments, the AA defines a universe of attributes Ω and a collision-resistant hash function $\mathcal{H} : \{0, 1\}^* \rightarrow \mathbb{Z}_q$. For each semantic attribute $attr_j \in \Omega$, the AA computes a deterministic identifier:

$$ID_j = \mathcal{H}(attr_j) \quad (1)$$

The AA executes lattice trapdoor generation to produce the Master Public Key (MPK) and Master Secret Key (MSK), binding public parameter matrices to the hashed identifiers:

$$(MPK, MSK) \leftarrow \text{Setup}(1^\lambda, \{ID_j\}_{j=1}^n) \quad (2)$$

To eliminate repeated policy expansion overhead, the access structure is precomputed during system initialization. Specifi-

cally, the policy tree TID is transformed into its Linear Secret Sharing Scheme (LSSS) representation and cached for reuse.

$$(M, \rho_{ID}) \leftarrow \text{PolicyTreeGen}(\mathbb{T}_{ID}) \quad (3)$$

$$\mathcal{C}_{policy}(TID) = (M, \rho_{ID}) \quad (4)$$

Authorized users are issued attribute-linked secret keys:

$$SK_u \leftarrow \text{KeyGen}(MSK, \{ID_j\}_{j \in Attr_u}) \quad (5)$$

Gateway Initialization. The gateway generate a CRYSTALS-Kyber key pair as follow:

$$(pk_{GW}, sk_{GW}) \leftarrow \text{KeyGen}_{Kyber}(1^\lambda) \quad (6)$$

where pk_{GW} denotes the public key, sk_{GW} denotes the corresponding secret key, 1^λ represents the security parameter of length λ , and KeyGen_{Kyber} is the key generation algorithm of the CRYSTALS-Kyber scheme.

The IIoT device derive a secret key and cipherkey using post-quantum key encapsulation every:

$$(cg_{kem,i}, kg_{kem,i}) \leftarrow \text{Encap}_{Kyber}(pk_{GW}) \quad (7)$$

Fog Node Initialization. Each fog node initializes its post-quantum communication capability by generating a CRYSTALS-Kyber key pair:

$$(pk_{FN}, sk_{FN}) \leftarrow \text{KeyGen}_{Kyber}(1^\lambda) \quad (8)$$

The public key pk_{FN} is distributed to registered IIoT devices for secure key establishment.

Kyber Precomputation. To accelerate lattice-based operations, each fog node precomputes the expanded public matrix and its Number Theoretic Transform (NTT) representation. This enables efficient polynomial operations during encapsulation and decapsulation.

$$A \leftarrow \text{ExpandA}(seed_{FN}) \quad (9)$$

$$\hat{A} = \text{NTT}(A) \quad (10)$$

$$\mathcal{C}_{Kyber}(F_j) = \hat{A} \quad (11)$$

To support hierarchical scheduling in Spider++, each fog node instantiates a set of enclaves:

$$F_j = \{E_{j,k} \mid k = 1, \dots, m_j\} \quad (12)$$

and initializes local telemetry monitors to track enclave-level states, including queue length, service rate, and EPC availability. These metrics are periodically reported to the Master Fog Node (MFN) to enable global scheduling decisions.

Policy Preparation. IIoT devices (or associated gateways) pre-compute hashed attribute identifiers and construct the access policy tree $\mathbb{T}_{ID}$ using $\{ID_j\}$, ensuring that no semantic attribute information is exposed during encryption and evaluation.

Phase II: Hardware-Bound IIoT Encryption: When device ID_i senses data m_i , it first derives a hardware-bound secret using a Physical Unclonable Function (PUF). A noisy response is obtained and reconstructed into a stable secret via a fuzzy extractor:

$$R_{noisy,i} \leftarrow f_{PUF}(C_i) \quad (13)$$

$$R_{secret,i} \leftarrow \text{FuzzyRep}(R_{noisy,i}, HD_i) \quad (14)$$

The device then performs post-quantum key encapsulation to the fog node public key:

$$(c_{kem,i}, k_{kem,i}) \leftarrow \text{Encap}_{Kyber}(pk_{FN}) \quad (15)$$

A device-bound session key is derived as:

$$K_{S,i} = \text{KDF}(H(k_{kem,i}) \parallel R_{secret,i} \parallel ID_i \parallel t_i \parallel ctr_i) \quad (16)$$

The associated authenticated data (AAD) is constructed as:

$$AAD_i = ID_i \parallel t_i \parallel ctr_i \quad (17)$$

The sensed data is encrypted using ChaCha20-Poly1305:

$$(CT_i, Tag_i) \leftarrow \text{ChaCha20-Poly1305.Enc}(K_{S,i}, N_i, m_i, AAD_i) \quad (18)$$

To enable gateway-side packet authentication and early filtering, the device computes a message authentication code using $kg_{kem,i}$:

$$Auth_i \leftarrow \text{MAC}_{kg_{kem,i}}(ID_i \parallel Metadata \parallel c_{kem,i} \parallel CT_i \parallel Tag_i \parallel t_i) \quad (19)$$

Finally, the device transmits the packet to the gateway:

$$P_i = \{ID_i, Metadata, c_{kem,i}, cg_{kem,i}, CT_i, N_i, AAD_i, Tag_i, Auth_i\} \quad (20)$$

Phase III: Gateway Validation and Micro-Batch Formation: Upon receiving packets from IIoT devices, the gateway performs authentication and integrity verification to filter out invalid or malicious inputs before forwarding data to the fog layer.

For each received packet P_i , the gateway recomputes the message authentication code using the derived kyber key:

$$kg_{kem,i} \leftarrow \text{Decap}_{Kyber}(cg_{kem,i}) \quad (21)$$

$$Auth'_i \leftarrow \text{MAC}_{kg_{kem,i}}(ID_i \parallel Metadata \parallel c_{kem,i} \parallel CT_i \parallel Tag_i \parallel t_i) \quad (22)$$

The packet is accepted only if $Auth'_i = Auth_i$. Otherwise, it is discarded. This ensures that only packets originating from legitimate devices with valid session keys are admitted into the system.

All validated packets collected within a fixed time window t_{window} are grouped into a micro-batch. Let $\mathcal{P}_{valid}$ denote the set of validated packets. To ensure batch-level integrity and prevent tampering, the gateway computes a group authentication tag:

$$GTag \leftarrow H(\text{Sort}(Auth_i \mid P_i \in \mathcal{P}_{valid}) \parallel t_G) \quad (23)$$

Algorithm 1 Gateway Validation and Micro-Batch Formation**Require:** Incoming packet stream during epoch t_{window} **Ensure:** Validated micro-batch B_k

```

1:  $\mathcal{P}_{valid} \leftarrow \emptyset$ 
2:  $\mathcal{M} \leftarrow \emptyset$ 
3: for all received packets  $P_i$  do
4:   Compute:
5:    $kg_{kem,i} \leftarrow Decap_{Kyber}(cg_{kem,i})$ 
6:    $Auth'_i \leftarrow MAC_{kg_{kem,i}} \left( ID_i \parallel Metadata \parallel c_{kem,i} \parallel \right)$ 
7:   if  $Auth'_i = Auth_i$  then
8:      $\mathcal{P}_{valid} \leftarrow \mathcal{P}_{valid} \cup \{P_i\}$ 
9:      $\mathcal{M} \leftarrow \mathcal{M} \cup \{Auth_i\}$ 
10:  else
11:    Discard  $P_i$ 
12:  end if
13: end for
14: if  $\mathcal{P}_{valid} = \emptyset$  then
15:   return  $\emptyset$ 
16: end if
17:  $t_G \leftarrow \text{GatewayTimestamp}()$ 
18: Compute:  $[GTag \leftarrow H(\text{Sort}(\mathcal{M}) \parallel t_G)]$ 
19:  $B_k \leftarrow BID, \mathcal{P}_{valid}, GTag, t_G$ 
20: return  $B_k$ 

```

where $\text{Sort}(\cdot)$ enforces deterministic ordering and t_G is the gateway timestamp.

The micro-batch is then constructed as:

$$B_k = BID, \mathcal{P}_{valid}, GTag, t_G \quad (24)$$

The validated batch B_k is forwarded to the Master Fog Node (MFN) for scheduling under Spider.

Remark: Under the hierarchical scheduling model introduced in Phase IV, the batch B_k may be processed as a whole or decomposed into sub-batches $B_k^{(1)}, \dots, B_k^{(r)}$ to enable enclave-parallel execution.

The complete validation and micro-batch formation procedure is summarized in Algorithm 1.

Phase IV: Spider Hierarchical Enclave-Oriented Load Balancing: This phase presents **Spider** (Secure Priority-Aware Intelligent Dual-Level Enclave Routing), a hierarchical scheduling framework that performs *security-aware*, *capability-aware*, and *enclave-parallel* workload orchestration in the fog layer.

Unlike conventional load balancing approaches that treat each fog node as a single execution unit, Spider++ explicitly models multiple TEEs (enclaves) within each fog node and performs two-level scheduling: (i) **inter-node routing** across fog nodes, and (ii) **intra-node enclave allocation** within each fog node.

This design enables fine-grained resource utilization, reduces EPC contention, and supports parallel execution of cryptographic workloads.

1) *Hierarchical Resource Model:* Each fog node is modeled as a set of enclaves:

$$F_j = E_{j,k} \mid k = 1, \dots, m_j \quad (25)$$

Each enclave maintains an independent state:

$$\Psi(E_{j,k}) = q_{j,k}, \mu_{j,k}, M_{j,k}^{free}, \rho_{j,k} \quad (26)$$

where $q_{j,k}$ denotes the queue length, $\mu_{j,k}$ is the service rate, $M^{free} * j, k$ represents available EPC memory, and $\rho * j, k$ captures runtime contention (e.g., cache or EPC pressure).

2) *Batch Profiling:* Upon receiving a validated batch B_k , the Master Fog Node (MFN) constructs a workload profile:

$$\Phi(B_k) = S_k, \omega_k, \eta_k, \delta_k \quad (27)$$

where: $S_k = |B_k|$ is batch size, ω_k is cryptographic workload intensity, η_k is security urgency, δ_k is deadline sensitivity.

The workload intensity is estimated as:

$$\omega_k = \beta_1 S_k + \beta_2 \cdot \text{AttrCount}(B_k) + \beta_3 \cdot \text{PolicyDepth}(B_k) \quad (28)$$

The urgency score is:

$$\eta_k = \gamma_1 \cdot \text{RiskScore}(B_k) + \gamma_2 \cdot \text{AnomalyFlag}(B_k) \quad (29)$$

Deadline sensitivity is:

$$\delta_k = \frac{1}{D_k - t_{\text{now}} + \epsilon} \quad (30)$$

The enclave memory requirement M_k^{req} is derived from ω_k via an offline-calibrated model.

3) *Fog Node State Model:* Each fog node periodically reports:

$$\Psi(F_j) = Q_j^{TEE}, Q_j^{REE}, \mu_j^{TEE}, \mu_j^{REE}, L_j, E_j, C_j, U_j \quad (31)$$

where Q_j^{TEE}, Q_j^{REE} denote queue lengths, μ_j^{TEE} and μ_j^{REE} denote service rates, L_j is network latency, E_j is normalized EPC availability, C_j is capability score, and U_j is trust-health score.

Level 1: Inter-Node Scheduling: The MFN selects the optimal fog node for batch B_k by computing and minimizing a Spider score.

Waiting Time:

$$T_{\text{wait}}(F_j) = \frac{Q_j^{TEE} + 1}{\mu_j^{TEE}} + \frac{Q_j^{REE} + 1}{\mu_j^{REE}} \quad (32)$$

EPC Pressure:

$$P_{\text{epc}}(F_j, B_k) = \lambda_1 \max \left(0, \frac{M_k^{req}}{E_j M_j^{total}} - \tau \right)^2 \quad (33)$$

Capability Penalty:

$$P_{\text{cap}}(F_j, B_k) = \lambda_2 \max(0, \omega_k - C_j) \quad (34)$$

Trust Penalty:

$$P_{\text{trust}}(F_j) = \lambda_3 (1 - U_j) \quad (35)$$

Spider Score:

$$\begin{aligned} \text{SpiderScore}(F_j, B_k) = & w_1 T_{\text{wait}}(F_j) + w_2 L_j + w_3 P_{\text{epc}} \\ & + w_4 P_{\text{cap}} + w_5 P_{\text{trust}} \\ & - w_6 \eta_k U_j - w_7 \delta_k \mu_j^{TEE} \end{aligned} \quad (36)$$

To further improve efficiency, Spider++ incorporates computation reuse awareness into scheduling decisions. Fog nodes that already cache relevant policy structures or cryptographic components are prioritized, reducing redundant computation and improving latency and energy efficiency.

$$I_{\text{policy}}(F_j, B_k) = \begin{cases} 1, & \text{if } TID_k \in \mathcal{C}_{\text{policy}}(F_j) \\ 0, & \text{otherwise} \end{cases} \quad (37)$$

$$I_{\text{kyber}}(F_j) = \begin{cases} 1, & \text{if } \mathcal{C}_{\text{Kyber}}(F_j) \text{ exists} \\ 0, & \text{otherwise} \end{cases} \quad (38)$$

$$R_{\text{reuse}}(F_j, B_k) = \theta_1 I_{\text{policy}}(F_j, B_k) + \theta_2 I_{\text{kyber}}(F_j) \quad (39)$$

$$\text{SpiderScore}'(F_j, B_k) = \text{SpiderScore}(F_j, B_k) - w_8 R_{\text{reuse}}(F_j, B_k) \quad (40)$$

The indicator functions I_{policy} and I_{kyber} capture whether a fog node has cached the required policy structure or precomputed cryptographic components. The reuse score R_{reuse} quantifies the availability of reusable computation, enabling the scheduler to prioritize nodes with lower expected processing overhead.

Node Selection:

$$F^* = \arg \min_{F_j \in F_{\text{feasible}}} \text{SpiderScore}'(F_j, B_k) \quad (41)$$

Level 2: Intra-Node Enclave Scheduling: After selecting F^* , Spider++ performs fine-grained enclave allocation.

Enclave Waiting Time:

$$T_{\text{wait}}(E_{j,k}) = \frac{q_{j,k} + 1}{\mu_{j,k}} \quad (42)$$

Enclave EPC Pressure:

$$P_{\text{epc}}(E_{j,k}, B_k) = \lambda'_1 \max \left(0, \frac{M_k^{\text{req}}}{M_{j,k}^{\text{free}}} - \tau \right)^2 \quad (43)$$

Contention Penalty:

$$P_{\text{cont}}(E_{j,k}) = \lambda'_2 \rho_{j,k} \quad (44)$$

Affinity Score:

$$A(E_{j,k}, B_k) = \begin{cases} 1, & \text{if similar workload was processed previously} \\ 0, & \text{otherwise} \end{cases} \quad (45)$$

Enclave Score:

$$\begin{aligned} \text{EnclaveScore}(E_{j,k}, B_k) = & z_1 T_{\text{wait}}(E_{j,k}) \\ & + z_2 P_{\text{epc}}(E_{j,k}, B_k) \\ & + z_3 P_{\text{cont}}(E_{j,k}) \\ & - z_4 A(E_{j,k}, B_k) \end{aligned} \quad (46)$$

Enclave Selection:

$$E^* = \arg \min_{E_{j,k} \in F^*} \text{EnclaveScore}(E_{j,k}, B_k) \quad (47)$$

4) *Parallel Batch Decomposition:* To exploit enclave-level parallelism and reduce end-to-end processing delay, the incoming batch is decomposed into multiple sub-batches:

$$B_k \rightarrow \{B_k^{(1)}, \dots, B_k^{(r)}\} \quad (48)$$

where r denotes the degree of parallelism, adaptively determined by the number of feasible enclaves, their current queue states, EPC availability, and the estimated workload intensity of B_k . Each sub-batch is independently scheduled to a selected enclave for concurrent execution. This decomposition allows Spider++ to balance cryptographic workloads across multiple TEEs, improve throughput, and mitigate enclave contention caused by assigning the entire batch to a single secure execution unit.

Each sub-batch is independently scheduled:

$$E^{*(i)} = \arg \min \text{EnclaveScore}(E_{j,k}, B_k^{(i)}) \quad (49)$$

This reduces processing latency and enables concurrent cryptographic operations.

5) *Admission Control:* A task is admitted only if:

$$M_{j,k}^{\text{free}} \geq \alpha M_k^{\text{req}} \quad (50)$$

6) *Stability Control:* To prevent oscillation:

$$\Delta \text{Score} \geq \zeta \quad (51)$$

7) *Secure Delegation:* If no feasible node exists, Spider++ performs secure delegation:

$$\Xi_k = ID_k, Meta_k, CT_k^{\text{partial}}, A_k, T_k, \sigma_k \quad (52)$$

Delegation uses post-quantum key encapsulation (Kyber), ensuring confidentiality of intermediate states.

The scheduling complexity is:

$$\mathcal{O}(|\mathcal{F}| + \sum_j m_j) \quad (53)$$

where $|\mathcal{F}|$ is the number of fog nodes and m_j is the number of enclaves per node.

Spider transforms conventional fog scheduling into a **hierarchical enclave-oriented orchestration framework** that jointly optimizes latency, security, and resource utilization. By integrating multi-enclave parallelism, EPC-aware scheduling, and trust-driven routing, the framework enables efficient and secure execution of cryptographic workloads in large-scale IIoT environments.

Phase V: Multi-Enclave TEE-Assisted Split-Phase CP-ABE Processing: After receiving the validated batch B_k , the selected fog node F^* performs secure batch processing using a two-level design: (i) enclave-parallel execution and (ii) split-phase CP-ABE encryption.

Step 1: Batch Integrity Verification (REE): The fog node verifies batch integrity by recomputing:

$$GTag' \leftarrow H(\text{Sort}(\text{Auth}_i \mid P_i \in \mathcal{P}_{\text{valid}}) \parallel t_G) \quad (54)$$

The batch is accepted only if $GTag' = GTag$.

Algorithm 2 Enclave-Parallel Processing at Fog Node**Require:** Batch B_k , enclave set $\mathcal{E}^*$ **Ensure:** (M_{agg}, AAD_{agg})

- 1: Verify GT_{ag}
- 2: **if** invalid **then return** $\perp$
- 3: **end if**
- 4: Partition $B_k \rightarrow B_k^{(i)}$
- 5: **for all** $B_k^{(i)}$ in parallel **do**
- 6: **for all** $P_\ell \in B_k^{(i)}$ **do**
- 7: Decrypt m_ℓ
- 8: **end for**
- 9: Aggregate $M_{agg}^{(i)}, AAD_{agg}^{(i)}$
- 10: **end for**
- 11: Merge all results
- 12: **return** (M_{agg}, AAD_{agg})

Step 2: Parallel Batch Decomposition: Based on Spider scheduling (Phase IV), the batch is either processed as a whole or decomposed:

$$B_k \rightarrow B_k^{(1)}, \dots, B_k^{(r)} \quad (55)$$

where r is determined by enclave availability and EPC constraints. Each sub-batch is assigned to an enclave $E^{*(i)}$.

Step 3: Enclave-Parallel Secure Processing (TEE): Each enclave processes its assigned sub-batch independently:

$$m_\ell \leftarrow \text{ChaCha20-Poly1305.Dec}(K_{S,\ell}, N_\ell, CT_\ell, AAD_\ell, Tag_\ell) \quad (56)$$

Local aggregation:

$$M_{agg}^{(i)} = m_1^{(i)} \parallel \dots \parallel m_{s_i}^{(i)} \quad (57)$$

$$AAD_{agg}^{(i)} = AAD_1^{(i)} \parallel \dots \parallel AAD_{s_i}^{(i)} \quad (58)$$

After all enclaves finish:

$$M_{agg} = \bigoplus_{i=1}^r M_{agg}^{(i)} \quad (59)$$

$$AAD_{agg} = \bigoplus_{i=1}^r AAD_{agg}^{(i)} \quad (60)$$

Insight: This design reduces decryption latency and avoids EPC bottlenecks.

Step 4: Symmetric Encryption within TEE: After enclave-parallel aggregation, the fog node protects the aggregated plaintext before applying attribute-based encryption. A fresh symmetric key is generated inside the enclave:

$$K_{AES} \leftarrow 0, 1^{256} \quad (61)$$

The aggregated data is then encrypted using authenticated encryption (AES-GCM):

$$(CT_{AES}, Tag_{AES}) \leftarrow \text{AES-GCM.Enc}(K_{AES}, IV, M_{agg}, AAD_{agg}) \quad (62)$$

This step ensures confidentiality and integrity of the aggregated data, while enabling efficient hybrid encryption.

Step 5: Partial CP-ABE Encryption within TEE: To securely encapsulate the symmetric key under an access policy, the enclave initiates a lattice-based CP-ABE encryption process. It samples secret and noise terms:

$$s, \mathbf{e}_0, \mathbf{e}_{base} \leftarrow \chi \quad (63)$$

and constructs a secret-sharing vector:

$$\mathbf{v} = (s, y_2, \dots, y_n)^\top \quad (64)$$

The core ciphertext component is computed as:

$$CT_0 = \mathbf{A}^\top s + \mathbf{e}_0 * K_{AES} \left\lfloor \frac{q}{2} \right\rfloor \pmod{q} \quad (65)$$

To enable efficient offloading of policy expansion, the enclave generates a blinded base vector:

$$\mathbf{V}_{base} = \mathcal{C}_{base}(TID) \cdot \mathbf{v} + \mathbf{e}_{base} \quad (66)$$

Base Vector Optimization. Since the matrix transpose is independent of runtime randomness, it is precomputed and cached per policy. This avoids repeated matrix transformations during encryption.

$$\mathcal{C}_{base}(TID) = \mathbf{A}^\top \quad (67)$$

Only non-sensitive intermediate values are exported to the REE:

$$\Psi_{out} = (CT_{AES}, Tag_{AES}, CT_0, \mathbf{V}_{base}, \mathbb{T}_{ID}, \mathcal{A}) \quad (68)$$

Remark: All sensitive values, including $s, \mathbf{v}, K_{AES}$, and M_{agg} , remain strictly inside the enclave and are erased after execution.

Step 6: CP-ABE Completion in REE: The REE completes the encryption by expanding the access policy. Instead of recomputing the policy structure, it retrieves the precomputed LSSS representation from cache:

$$(M, \rho_{ID}) \leftarrow \mathcal{C}_{policy}(TID) \quad (69)$$

For each row i , the ciphertext components are computed as:

$$CT_i = M_i \cdot \mathbf{V}_{base} + \mathbf{A}_{\rho_{ID}(i)}^\top \mathbf{r}_i + \mathbf{e}_{1,i} \pmod{q} \quad (70)$$

The final CP-ABE ciphertext is constructed as:

$$CT_{L-ABE} = (CT_0, (M, \rho_{ID}), CT_{i=1}^\ell) \quad (71)$$

This split-phase design ensures that sensitive cryptographic operations are confined within the TEE, while computationally intensive matrix operations are efficiently handled by the REE.

Step 7: Signature and Upload:

$$\sigma \leftarrow \text{Sign}_{\text{Dilithium}}(sk_{FN}, H(BID \parallel CT_{AES} \parallel CT_{L-ABE})) \quad (72)$$

$$\Omega = BID, CT_{AES}, Tag_{AES}, IV, CT_{L-ABE}, \sigma \quad (73)$$

Final Remark: Although execution is parallelized across enclaves, the system outputs a single ciphertext Ω , ensuring compatibility with standard CP-ABE decryption.

Phase VI: User Verification and Decryption: An authorized user retrieves the encrypted package Ω from cloud storage.

Step 1: Integrity and Authenticity Verification: The user first verifies the integrity and origin of the data using the fog node's public key:

$$\text{Verify}_{\text{Dilithium}}(pk_{FN}, \sigma, H(BID \parallel CT_{AES} \parallel CT_{L-ABE})) \quad (74)$$

If verification fails, the ciphertext is rejected.

Step 2: Policy Satisfaction Check: Let (M, ρ_{ID}) be the LSSS representation embedded in CT_{L-ABE} . The user checks whether their attribute set $Attr_u$ satisfies the access policy.

If satisfied, the user derives reconstruction weights ω_i such that:

$$\sum_{i \in I} \omega_i M_i = (1, 0, \dots, 0) \quad (75)$$

Step 3: Ciphertext Share Evaluation: Using the attribute-based secret key SK_u , the user evaluates:

$$C'_i \leftarrow \text{Eval}(CT_i, SK_u) \approx M_i \cdot \mathbf{V}_{base} + e_{eval,i} \quad (76)$$

Step 4: Secret Reconstruction: The user reconstructs the blinded vector:

$$C_{rec} = \sum_{i \in I} \omega_i C'_i \approx \mathbf{A}^T s + e_{total} \quad (77)$$

Step 5: Symmetric Key Recovery: The symmetric key is recovered by:

$$K_{AES} \leftarrow \text{Decode}(\lfloor CT_0 - C_{rec} \rfloor_2) \quad (78)$$

Step 6: Final Decryption: The aggregated plaintext is recovered as:

$$M_{agg} \leftarrow \text{AES-GCM.Dec}(K_{AES}, IV, CT_{AES}, Tag_{AES}) \quad (79)$$

This ensures that only users whose attributes satisfy the access policy can recover the underlying IIoT data.

IV. FAULT TOLERANCE: ENCLAVE-AWARE, SECURITY-AWARE, AND STABILITY-AWARE LEADER ELECTION

To support hierarchical scheduling in Spider, we propose an *enclave-aware, security-aware, and stability-aware leader election mechanism* for dynamically selecting the Master Fog Node (MFN). Unlike conventional leader election approaches that rely on coarse-grained resource metrics (e.g., CPU or memory), the proposed mechanism explicitly incorporates intra-node enclave dynamics, execution trustworthiness, and temporal stability. This design is particularly critical in IIoT environments, where workloads are both security-sensitive and highly dynamic, and where each fog node hosts multiple concurrent TEE enclaves with heterogeneous execution states.

Capability and Security-Aware Scoring

Each fog node F_j computes a composite capability score:

$$\text{Score}(F_j) = \alpha_1 C_j + \alpha_2 M_j - \alpha_3 N_j + \alpha_4 R_j + \alpha_5 T_j \quad (80)$$

where C_j and M_j denote available computational and memory resources, respectively, N_j represents network latency to the gateway or peer nodes, R_j captures system readiness (defined below), T_j denotes a trust score reflecting secure execution behavior.

The trust component T_j is derived from runtime security indicators such as enclave attestation success rate, absence of abnormal execution patterns, and historical integrity verification outcomes. By integrating T_j , the system ensures that leadership is not assigned to nodes that, while resource-rich, may exhibit suspicious or degraded behavior.

Enclave-Aware Node Readiness

To accurately reflect the internal state of fog nodes, readiness is computed by incorporating both inter-node and intra-node conditions:

$$R_j = 1 - \left(\beta_1 \cdot \frac{1}{m_j} \sum_{k=1}^{m_j} \frac{Q_{j,k}}{Q_{\max}} \cdot \beta_2 \cdot \rho_{epc}(F_j) \cdot \beta_3 \cdot \frac{Q_j^{\text{REE}}}{Q_{\max}^{\text{REE}}} \right) \quad (81)$$

where: $Q_{j,k}$ is the queue length of enclave $E_{j,k}$, m_j is the number of enclaves in node F_j , $\rho_{epc}(F_j)$ captures EPC memory pressure, Q_j^{REE} represents backlog in the untrusted execution environment

Unlike traditional node-level metrics, this formulation captures *fine-grained load imbalance across enclaves*, enabling more accurate estimation of a node's ability to coordinate and process security-critical workloads. This is essential in Spider++, where scheduling decisions are tightly coupled with enclave availability and EPC constraints.

Stability-Aware Leader Selection

To prevent frequent leader oscillations under fluctuating workloads, we introduce a stability-aware selection mechanism:

$$MFN = \arg \max_{F_j \in \mathcal{F}_{candidate}} (\text{Score}(F_j) - \gamma \cdot \Delta \text{Score}(F_j)) \quad (82)$$

where $\Delta \text{Score}(F_j)$ captures short-term variations in node capability. This penalty term discourages selecting nodes with unstable performance, thereby ensuring continuity in control-plane operations and reducing coordination overhead.

Candidate nodes are defined as:

$$F_j \in \mathcal{F}_{candidate} \quad \text{if} \quad R_j \geq \tau \quad (83)$$

ensuring that only sufficiently stable and lightly loaded nodes participate in leader election.

Once selected, the MFN transitions into a dedicated control-plane role. It offloads its residual workloads to neighboring fog nodes and coordinates global scheduling using Spider++, including both inter-node workload distribution and intra-node enclave assignment. This separation between control-plane and data-plane execution prevents bottlenecks and enhances scalability.

From a fault-tolerance perspective, the proposed mechanism enables rapid recovery under node failure or degradation. If the current MFN becomes unavailable or its readiness drops below the threshold τ , a new leader is elected dynamically based on updated system states. The integration of enclave-level awareness, trust evaluation, and stability control ensures that leadership transitions remain both secure and efficient.

By jointly optimizing for resource capability, enclave-level dynamics, execution trust, and temporal stability, the proposed leader election mechanism provides a fundamentally stronger foundation for coordinating secure, high-throughput IIoT workloads in heterogeneous fog environments.

V. SECURITY ANALYSIS

In this section, we provide a formal security analysis of PQ-SPIDER under a post-quantum adversarial model. The analysis covers confidentiality of outsourced IIoT content, bounded policy privacy, bounded scheduler leakage, end-to-end integrity and authenticity, and forward secrecy of session-level data protection.

A. Security Goals

The security objectives of PQ-SPIDER are summarized as follows:

- **Confidentiality of outsourced IIoT data:** An adversary should learn no information about the plaintext micro-batch contents beyond explicitly modeled leakage.
- **Policy privacy up to structural leakage:** The adversary may observe the size and structural shape of the access policy, but not the semantic meaning of attributes.
- **Bounded scheduler leakage:** Dynamic routing under Spider++ should not reveal plaintext or secret-dependent information beyond coarse workload abstractions needed for scheduling.
- **Integrity and authenticity:** Forged device packets, modified outsourced ciphertexts, or forged fog-side outputs should be rejected except with negligible probability.
- **Forward secrecy of session protection:** Compromise of later system states should not expose previously established device-to-fog session keys.
- **Post-quantum resistance:** The above guarantees must hold against quantum polynomial-time adversaries under the assumed hardness of the underlying post-quantum primitives.

B. Adversarial Model and Trust Assumptions

We consider a probabilistic polynomial-time adversary $\mathcal{A}$ with full control over public communication channels. The following trust assumptions are adopted.

- The **Attribute Authority (AA)** is fully trusted and correctly generates the public parameters and master secret key.
- The **cloud** is honest-but-curious: it stores ciphertexts faithfully but may attempt to infer information from them.
- Fog-node **REE environments** are not trusted with plaintexts or secret keys.
- **TEEs** provide confidentiality and integrity for enclave-resident code and data.

C. Leakage-Aware Formal Security Model

At epoch e , the validated micro-batch is

$$B_e = \{P_{e,1}, \dots, P_{e,n_e}\} \quad (84)$$

where each packet is generated in Phases II–III. The outsourced object is

$$\Omega_e = (BID_e, CT_e^{AES}, Tag_e^{AES}, IV_e, CT_e^{L-ABE}, \sigma_e) \quad (85)$$

with plaintext batch

$$M_e = (m_{e,1}, \dots, m_{e,n_e}) \quad (86)$$

and access policy A_e . Over q epochs:

$$Hist_q = \{(M_e, A_e, \Omega_e)\}_{e=1}^q \quad (87)$$

We define leakage profile

$$\mathcal{L} = (L_{\text{setup}}, L_{\text{enc}}, L_{\text{sched}}, L_{\text{ret}}) \quad (88)$$

1) *Setup leakage:*

$$L_{\text{setup}}(1^\lambda) = (\lambda, |\Omega|, q, |PP|) \quad (89)$$

revealing only public parameters and system size.

2) *Encryption/outsourcing leakage:*

$$L_{\text{enc}}(M_e, A_e) = (n_e, |CT_e^{AES}|, |CT_e^{L-ABE}|, \ell(A_e), \tau(A_e), t_e) \quad (90)$$

where n_e is batch size, ciphertext sizes capture payload and header, $\ell(A_e)$ and $\tau(A_e)$ denote policy structure, and t_e is coarse timing. No plaintexts, attributes, keys, or enclave states are revealed.

3) *Scheduler leakage:*

$$L_{\text{sched}}(B_e) = (\Phi(B_e), j_e^*, r_e) \quad (91)$$

where $\Phi(B_e)$ is a coarse workload class, j_e^* the selected fog node, and r_e the parallelism factor. Only abstract workload and scheduling metadata are exposed.

4) *Retrieval leakage:*

$$L_{\text{ret}}(e, u) = (e, \text{acc}(u, e), |M_e|) \quad (92)$$

where $\text{acc}(u, e) \in \{0, 1\}$ indicates policy satisfaction.

5) *Total leakage*:

$$\mathcal{L}(\text{Hist}_q) = \left(L_{\text{setup}}, \{L_{\text{enc}}\}_{e=1}^q, \{L_{\text{sched}}\}_{e=1}^q, \{L_{\text{ret}}\}_{(e,u) \in Q_{\text{ret}}} \right) \quad (93)$$

D. Adaptive Leakage-Aware Indistinguishability

Definition 1 (Leakage-Aware Security Experiment). Let Π denote PQ-SPIDER. The experiment $\text{Expt}_{\Pi, \mathcal{A}}^{\text{LA-IND}}(\lambda)$ proceeds:

- 1) **Setup**: $(PP, MSK) \leftarrow \text{Setup}(1^\lambda)$; give PP to $\mathcal{A}$
- 2) **Queries**:
 - Key: $SK_u \leftarrow \text{KeyGen}(MSK, \text{Attr}_u)$ (non-authorizing)
 - Outsource: $(M_e, A_e) \mapsto \Omega_e$
 - Retrieve: return decryption or $\perp$
 - Scheduler: return L_{sched} only
- 3) **Challenge**: Submit $\text{Hist}_0^*, \text{Hist}_1^*$ such that

$$L_{\text{enc}}(M_{e,0}^*, A_{e,0}^*) = L_{\text{enc}}(M_{e,1}^*, A_{e,1}^*) \quad (94)$$

Challenger returns Ω_e from Hist_b^* , $b \leftarrow \{0, 1\}$

- 4) **Post-queries**: Same restrictions apply
- 5) **Guess**: Output b'

Definition 2 (Leakage-Aware Security). Π is $\mathcal{L}$ -secure if for all QPT adversaries $\mathcal{A}$

$$\text{Adv}_{\Pi, \mathcal{A}}^{\text{LA-IND}}(\lambda) = \left| \Pr[\text{Expt} = 1] - \frac{1}{2} \right| \leq \text{negl}(\lambda) \quad (95)$$

E. Simulation-Based Formulation

The above indistinguishability definition admits an equivalent simulation-based interpretation.

Definition 3 (Simulation-Based Leakage Security). PQ-SPIDER is simulation-secure with leakage $\mathcal{L}$ if there exists a PPT simulator $\mathcal{S}$ such that, for every PPT distinguisher $\mathcal{D}$

$$|\Pr[\mathcal{D}(\text{View}_{\text{real}}) = 1] - \Pr[\mathcal{D}(\text{View}_{\text{ideal}}) = 1]| \leq \text{negl}(\lambda) \quad (96)$$

where:

$$\text{View}_{\text{real}} = (PP, \{\Omega_e\}, \text{oracle transcripts}, \text{scheduling metadata}) \quad (97)$$

$$\text{View}_{\text{ideal}} \leftarrow \mathcal{S}(\mathcal{L}(\text{Hist}_q)) \quad (98)$$

The simulator receives only the leakage tuple and generates:

- Random strings matching the lengths of device-side ciphertexts and AES payloads,
- A simulated CP-ABE header matching the required policy dimensions and structural shape,
- Scheduler-visible outputs consistent with $\Phi(B_e)$, j_e^* , and r_e
- Retrieval transcripts revealing only success/failure and output length information.

F. Bounded Scheduler Leakage

Since Spider++ is a key architectural component, its leakage must be formally bounded.

Lemma 1 (Bounded Scheduler Leakage). The scheduler-visible information

$$L_{\text{sched}}(B_e) = (\Phi(B_e), j_e^*, r_e)$$

does not reveal plaintext contents, semantic attributes, session keys, or enclave-resident secrets beyond the defined leakage abstraction.

Proof. We show each component of L_{sched} is independent of confidential data.

(1) **Workload Abstraction** $\Phi(B_e)$ Let $\Phi : \mathcal{B} \rightarrow \mathcal{C}$ with $|\mathcal{C}| \ll |\mathcal{B}|$. Then

$$|\{B \in \mathcal{B} : \Phi(B) = c\}| \gg 1$$

so Φ is many-to-one and non-invertible. Thus, $\Phi(B_e)$ reveals only coarse workload properties.

(2) **Node Selection** j_e^*

$$j_e^* = \arg \min_j f(\Psi(F_j), \Phi(B_e))$$

where $\Psi(F_j)$ denotes system state. Since f depends only on (Ψ, Φ) , j_e^* is conditionally independent of plaintexts and secrets given L_{sched} .

(3) **Parallelism Factor** r_e

$$r_e = g(\Phi(B_e), \{\Psi(E_{j,k})\})$$

which depends only on workload class and enclave resources, not on encrypted data or attributes.

(4) **Secret Independence** The scheduler operates outside TEEs and accesses no sensitive variables:

$$K_{\text{AES}}, s, v, R_{\text{secret}}, K_{S,i}, m_{e,i}$$

Hence, for any secret X ,

$$\Pr[\mathcal{A} \text{ learns } X \mid L_{\text{sched}}] = \Pr[\mathcal{A} \text{ learns } X]$$

Therefore, all components of L_{sched} are either many-to-one abstractions or functions of system state, implying bounded leakage with no additional exposure of confidential information. $\square$

G. Main Leakage-Aware Security Theorem

Theorem 1 (Leakage-Aware Confidentiality of PQ-SPIDER). Assume: (i) Kyber is IND-CCA secure; (ii) KDF is a quantum-secure PRF; (iii) ChaCha20-Poly1305 and AES-GCM are IND-CPA and INT-CTXT secure; (iv) lattice CP-ABE is selectively secure under MLWE; (v) Dilithium is EUF-CMA secure; and (vi) the TEE provides ideal confidentiality and integrity. Then PQ-SPIDER is $\mathcal{L}$ -secure.

Proof. We prove via hybrid games.

G_0 (Real): Adversary receives real outsourced objects Ω_e from Hist_b^* .

G_1 (Randomize session keys): Replace

$$K_{S,i} = KDF(H(k_{kem,i}) \parallel R_{secret,i} \parallel ID_i \parallel t_i \parallel ctr_i)$$

with $\tilde{K}_{S,i} \leftarrow \{0,1\}^\kappa$. Then

$$|\Pr[G_0] - \Pr[G_1]| \leq Adv_{KDF}^{PRF}(\lambda)$$

G_2 (Randomize Kyber secrets): Replace $k_{kem,i}$ with uniform strings. Then

$$|\Pr[G_1] - \Pr[G_2]| \leq Adv_{Kyber}^{IND-CCA}(\lambda)$$

G_3 (Randomize device AEAD): Replace device ciphertexts with encryptions of zeros. Then

$$|\Pr[G_2] - \Pr[G_3]| \leq Adv_{ChaCha20-Poly1305}^{AEAD}(\lambda)$$

G_4 (Randomize AES payload): Replace $(CT_e^{AES}, Tag_e^{AES})$ with encryption of zeros:

$$|\Pr[G_3] - \Pr[G_4]| \leq Adv_{AES-GCM}^{AEAD}(\lambda)$$

G_5 (Simulate CP-ABE header): Replace CT_e^{L-ABE} with simulated values using only $(|CT|, \ell(A_e), \tau(A_e))$:

$$|\Pr[G_4] - \Pr[G_5]| \leq Adv_{L-CP-ABE}^{SEL-CPA}(\lambda)$$

Remark: Security is selective due to CP-ABE.

G_6 (Simulate signatures): Replace σ_e with random strings:

$$|\Pr[G_5] - \Pr[G_6]| \leq Adv_{Dilithium}^{EUF-CMA}(\lambda)$$

G_7 (Ideal): All outputs are generated from leakage $\mathcal{L}$: ciphertexts, headers, and signatures are simulated; scheduler and retrieval reveal only bounded leakage.

Thus,

$$\begin{aligned} Adv_{\Pi,A}^{LA-IND}(\lambda) &\leq Adv_{KDF}^{PRF} + Adv_{Kyber}^{IND-CCA} \\ &\quad + Adv_{ChaCha20-Poly1305}^{AEAD} + Adv_{AES-GCM}^{AEAD} \\ &\quad + Adv_{L-CP-ABE}^{SEL-CPA} + Adv_{Dilithium}^{EUF-CMA} + \text{negl}(\lambda) \end{aligned} \quad (99)$$

All terms are negligible; hence PQ-SPIDER is $\mathcal{L}$ -secure. $\square$

H. Integrity and Authenticity

PQ-SPIDER ensures integrity and authenticity across device transmission, fog processing, and cloud storage.

Theorem 2 (End-to-End Integrity and Authenticity). *Assume the device MAC is secure, ChaCha20-Poly1305 and AES-GCM satisfy ciphertext integrity, and Dilithium is EUF-CMA secure. Then any PPT adversary that forges a valid packet, modifies ciphertext undetected, or produces a valid forged fog output succeeds with negligible probability.*

Proof. We consider three layers.

(1) **Device Authenticity:** Each packet includes

$$Auth_i = MAC_{K_{S,i}}(ID_i \parallel Metadata \parallel c_{kem,i} \parallel CT_i \parallel Tag_i \parallel t_i)$$

A valid forgery without $K_{S,i}$ breaks MAC unforgeability.

(2) **Ciphertext Integrity:** Device data (ChaCha20-Poly1305) and fog payload (AES-GCM) are AEAD-protected. Any undetected modification implies breaking INT-CTXT security.

(3) **Fog Authenticity:** Users verify

$$\text{Verify}_{pk_{FN}}(\sigma_e, H(BID_e \parallel CT_e^{AES} \parallel CT_e^{L-ABE}))$$

A valid forgery implies breaking Dilithium EUF-CMA security.

By union bound,

$$\begin{aligned} Adv_{\Pi,A}^{\text{forge}}(\lambda) &\leq Adv^{\text{MAC}} + Adv_{ChaCha20-Poly1305}^{\text{INT-CTXT}} \\ &\quad + Adv_{AES-GCM}^{\text{INT-CTXT}} + Adv_{Dilithium}^{\text{EUF-CMA}} \end{aligned} \quad (100)$$

which is negligible. $\square$

I. Policy Privacy

Theorem 3 (Policy Privacy up to Structural Leakage). *Assuming hash collision resistance and selective security of lattice CP-ABE, the adversary learns only policy size and structure, not attribute semantics.*

Proof. Attributes are encoded as

$$ID_j = H(attr_j)$$

hiding semantic labels. The ciphertext reveals only $(\ell(A_e), \tau(A_e))$. Under CP-ABE selective security, no additional information is leaked beyond this structure. Hence, attribute semantics remain hidden. $\square$

J. Forward Secrecy of Device-to-Fog Session Protection

Session keys are derived as

$$K_{S,i} = KDF(H(k_{kem,i}) \parallel R_{secret,i} \parallel ID_i \parallel t_i \parallel ctr_i) \quad (101)$$

Theorem 4 (Forward Secrecy of Session Keys). *Assume Kyber is IND-CCA secure and KDF is pseudorandom. Then compromise of later system states does not reveal any past session key except with negligible probability.*

Proof. Let i^* be a completed session. The adversary $\mathcal{A}$ obtains all transcripts, including c_{kem,i^*} , and arbitrary later system states, but not the decapsulated secret k_{kem,i^*} inside the TEE.

In the indistinguishability experiment, the challenger returns

$$K^* = \begin{cases} K_{S,i^*}, & b = 0, \\ U_\kappa, & b = 1, \end{cases} \quad b \leftarrow \{0,1\}$$

Game G_0 : Real key

$$K_{S,i^*} = KDF(H(k_{kem,i^*}) \parallel \dots)$$

Game G_1 : Replace k_{kem,i^*} with $\tilde{k} \leftarrow \{0,1\}^\ell$:

$$|\Pr[G_0] - \Pr[G_1]| \leq Adv_{Kyber}^{IND-CCA}$$

Game G_2 : Replace KDF output with U_κ :

$$|\Pr[G_1] - \Pr[G_2]| \leq Adv_{KDF}^{PRF}$$

In G_2 , K^* is uniform, so $\Pr[G_2 = 1] = \frac{1}{2}$

TABLE III
NOTATION FOR COMPUTATION COST ANALYSIS

Symbol	Description
<i>System Parameters</i>	
$ F $	Number of fog nodes
m_j	Number of enclaves in fog node F_j
$ S $	Number of attributes in access policy
N	Input data size
<i>Lattice / CP-ABE Operations</i>	
T_{poly}	Polynomial (NTT-based) operation cost
T_{PRF}	Pseudorandom function cost
$T_{\text{L-ABE.Setup}}$	CP-ABE setup cost
$T_{\text{L-ABE.Enc}}$	CP-ABE encryption cost
$T_{\text{L-ABE.Dec}}$	CP-ABE decryption cost
<i>Post-Quantum Primitives</i>	
T_{Kyber}	Kyber encapsulation/decapsulation cost
T_{Dil}	Dilithium signing/verification cost
<i>Symmetric Operations</i>	
T_{AE}	AEAD encryption/decryption cost
T_{MAC}	MAC generation/verification cost
T_{PUF}	PUF evaluation cost
<i>Scheduling (Ours)</i>	
T_{sched}	Scheduling overhead
$\sum_j m_j$	Total available enclaves
$\frac{ S }{\sum_j m_j}$	Parallel workload factor

Thus,

$$\text{Adv}_{\Pi, \mathcal{A}}^{\text{fs}} \leq \text{Adv}_{\text{Kyber}}^{\text{IND-CCA}} + \text{Adv}_{\text{KDF}}^{\text{PRF}} \leq \text{negl}(\lambda)$$

Fresh $(c_{\text{kem}, i}, t_i, \text{ctr}_i)$ ensures independence across sessions, completing the proof. $\square$

VI. EVALUATION

This section presents the evaluation of the scheme by illustrating the functionality comparison and demonstrating the experiments conducted for measuring the processing time of encryption, decryption and load balancing.

A. Computation Cost Analysis

To provide a fair and structured evaluation, we divide the computation cost analysis into two groups: (i) post-quantum cryptographic protection schemes and (ii) fog load-balancing mechanisms. This separation is necessary because existing works typically focus on either cryptographic protection or scheduling efficiency, whereas the proposed framework integrates both aspects.

1) Post-Quantum Protection Cost Comparison. We compare our scheme with representative post-quantum protection works [4], [35], and [36], focusing on key generation, device/edge encryption, fog-side processing, and user decryption. Table III summarizes the notations used in the computation cost analysis.

Table IV presents a comparative analysis of the proposed scheme against representative approaches, highlighting differences in computational complexity, system design, and scalability.

Scheme [4] incurs relatively high overhead due to its reliance on lattice-based CP-ABE with fine-grained access

policies. The key generation cost scales with the number of attributes, while encryption and decryption involve multiple polynomial operations, Gaussian sampling, and modular arithmetic. As a result, its computational cost is strongly attribute-dependent, leading to increased latency, especially in large-scale deployments. Moreover, authentication and verification operations are embedded across device and fog layers, further contributing to the overall processing burden.

Scheme [35] adopts standalone post-quantum cryptographic primitives, where the computational cost is dominated by Kyber encapsulation/decapsulation and Dilithium signature generation. Each operation incurs a constant-time complexity, i.e., $O(1)$ per invocation. However, the scheme lacks integrated access control and distributed processing capabilities, limiting its scalability and making it less suitable for large-scale IIoT environments with high data volume and concurrency.

Scheme [36] follows a transformation-based design using chaotic-map encoding mechanisms. Its computational cost is dominated by iterative transformation and mapping processes, resulting in an overall complexity of $O(N^2)$ for encryption and decryption. Unlike structured or parallelizable designs, this quadratic dependence on data size significantly limits scalability as the dataset grows. Furthermore, the absence of distributed processing or workload partitioning makes it less adaptable to dynamic IIoT environments.

In contrast, the proposed scheme achieves a scalable and efficient computation model by combining lightweight device-side operations with fog-assisted parallel processing. At the device layer, the cost is dominated by efficient primitives such as Kyber encapsulation, ChaCha20-Poly1305 encryption, and PUF-based authentication, ensuring low overhead suitable for resource-constrained IIoT devices. At the fog layer, computationally intensive tasks, including batch decryption, CP-ABE encryption, and signature generation, are offloaded and parallelized across multiple enclaves.

As shown in Table IV, the fog-side complexity is effectively reduced by a factor proportional to $\frac{|B_k|}{\sum_j m_j}$ for batch processing and $\frac{|S|}{\sum_j m_j}$ for attribute-based encryption, where $|B_k|$ denotes the batch size and $\sum_j m_j$ represents the total number of available enclaves. This parallel execution model significantly lowers processing latency and enables horizontal scalability.

Overall, unlike schemes with attribute-dependent overhead [4], limited system integration [35], or quadratic data-dependent complexity [36], the proposed framework provides a unified, distributed, and scalable computation model that is well-suited for large-scale and latency-sensitive IIoT environments.

2) Load Balancing Cost Analysis

We analyze the computational and communication overhead of load balancing mechanisms across representative fog/edge scheduling schemes. Table V summarizes the notations used in this analysis.

Table VI compares the LB strategies and associated costs of representative schemes. Since the compared works adopt

TABLE IV
COMPUTATION COST COMPARISON

Scheme	KeyGen	Encrypt (Edge)	Encrypt (Fog)	Decrypt
[4]	$(N_{\text{strict}} + N_{\text{flexible}} + 1)H$ + $N_{\text{flexible}} \text{PRF}$ + $CD_{\text{keygen}} + \text{EpochInit}$	AES-GCM + $(N_a + 3)SG$ + $(N_a + 2)Mul_R$ + $(N_a + 3)Add_b + N_a H$ + MAC_{gen}	$MAC_{\text{ver}} + F Mul_R$ + $Add_R + Sign$	<i>Device:</i> $MAC_{\text{ver}} + N_{\text{strict_used}} Mul_R$ + $(2N_{\text{strict_used}} + 2)Add_R$ + $H + T_{\text{decode}}$ + AES-GCM <i>Fog:</i> $Ver + N_{\text{flex_used}} Mul_R$ + $Add_R + MAC_{\text{gen}}$
[35]	$O(T_{\text{KyberKG}} + T_{\text{Dil.KG}})$	$O(T_{\text{KyberEnc}} + T_{\text{Dil.Sign}})$	–	$O(T_{\text{KyberDec}})$
[36]	–	$O(N^2)$ (Chaotic-map Based)	–	$O(N^2)$
Ours	$O(T_{\text{KyberKG}} + T_{\text{L-ABE.Setup}})$	$O(T_{\text{PUF}} + T_{\text{KyberEnc}} + T_{\text{ChaChaEnc}} + T_{\text{MAC}})$	$O\left(\frac{ B_k }{\sum_j m_j} \cdot (T_{\text{KyberDec}} + T_{\text{ChaChaDec}})\right)$ + $O(T_{\text{AESEnc}})$ + $O\left(\frac{ S }{\sum_j m_j} \cdot T_{\text{L-ABE.Enc}}\right)$ + $O(T_{\text{Dil.Sign}} + T_{\text{sched}})$	$O(T_{\text{L-ABE.Dec}} + T_{\text{AESDec}})$

TABLE V
NOTATION FOR LOAD BALANCING COST ANALYSIS

Symbol	Description
$ F $	Number of fog nodes
m_j	Number of enclaves in fog node F_j
T	Number of incoming tasks
N_{usage}	Total network traffic volume
$ MC $	Number of edge/medical centers
P	Number of workload requests/tasks
R	Number of regions
I	Number of learning iterations
d	Average neighbor degree

different architectures, we report dominant scheduling and communication costs derived from their algorithms.

Existing schemes generally rely on single-level or iterative scheduling mechanisms with varying computational overhead. Scheme [22] assigns each incoming task by scanning all candidate fog nodes and selecting the minimum-latency node, leading to a scheduling cost of $O(T|F|)$. Its communication overhead depends on total network traffic, modeled as $O(N_{\text{usage}})$, which grows with system workload and inter-layer data exchange.

Scheme [37] improves adaptability by combining static planning and SDN-assisted dynamic control. However, its scheduling cost is dominated by pairwise coordination and workload distribution across centers, resulting in $O(|MC|^2 + |MC| + P)$. The communication overhead scales with task distribution and inter-node coordination, captured as $O(Pd)$.

Scheme [39] adopts a distributed reinforcement learning-based scheduling mechanism. Its dominant cost arises from iterative Q-learning across regions, fog nodes, and tasks, yielding a complexity of $O(RI|F|T)$. Additionally, communication overhead is introduced through Q-value sharing among neighboring regions, modeled as $O(RId)$.

In contrast, the proposed scheme introduces a hierarchical and adaptive LB mechanism that jointly considers fog nodes and enclave-level resources. The scheduling complexity is

reduced to $O(|F| + \sum_j m_j)$, where $\sum_j m_j$ denotes the total number of enclaves across fog nodes. The communication overhead follows the same bound, as only lightweight state information and scheduling decisions are exchanged. This two-level design enables fine-grained parallelism across enclaves, significantly improving scalability and reducing bottlenecks compared to single-level or learning-based approaches.

B. Performance Analysis

This section evaluates the performance of the proposed scheme in terms of computational overhead and load-balancing efficiency.

1) *Computation Cost Comparison:* To analyze the cryptographic cost of the proposed framework, computation overhead is measured at each major processing stage. The comparison is illustrated through the following graphs:

- 1) Key Generation Graph
- 2) Encryption at the Edge Graph
- 3) Encryption at the Fog Graph
- 4) Decryption Graph

2) *Load-Balancing Cost Analysis:* To evaluate the effectiveness of the proposed dynamic load-balancing mechanism, the following performance metrics are considered:

- 1) Latency Comparison
- 2) Execution Time
- 3) Energy Consumption
- 4) Network Usage

VII. CONCLUSION

REFERENCES

- [1] S. Fugkeaw, A. Changtor, T. Maneerat, P. Rattanasrisuk, and K. Tangtanawirut, "LightPUF-IIoT: A Lightweight PUF-Based Authentication Scheme With Real-Time Detection of Rogue Devices in Fog-Assisted IIoT Data Sharing," *IEEE Open Journal of the Computer Society*, vol. 6, pp. 1438–1450, 2025, doi: 10.1109/OJCS.2025.3607984.

TABLE VI
LOAD BALANCING COST COMPARISON

Scheme	LB Strategy	Scheduling Cost	Communication Cost	Scalability / Scope
[22]	Optimised Load Balancing (OLB)	$O(T F)$	$O(N_{usage})$	Single-level Fog Scheduling
[37]	Adaptive SDN-based Load Balancing	$O(MC ^2 + MC + P)$	$O(Pd)$	Multi-tier Edge/Fog with SDN Controller
[39]	Distributed Reinforcement Learning-based Task Scheduling	$O(RI F T)$	$O(RId)$	Multi-round Distributed Learning-based Scheduling
Ours	Hierarchical Adaptive Fog and Enclave Scheduling	$O(F + \sum_j m_j)$	$O(F + \sum_j m_j)$	Two-level Scheduling Fog Layer and Enclave

- [2] A. Changtor, T. Maneerat, P. Rattanasrisuk, K. Tangtanawirut, and S. Fugkeaw, "A Secure and Lightweight IIoT Data Authentication Using PUF-Based in IoT Fog-Assisted Cloud," in *Proc. 2025 17th International Conference on Knowledge and Smart Technology (KST)*, 2025, pp. 40–45, doi: 10.1109/KST65016.2025.11003299.
- [3] Y. Yang, J. Zhang, X. Liu, and J. Ma, "A Scalable and Auditable Secure Data Sharing Scheme With Traceability for Fog-Based Smart Logistics," *IEEE Internet of Things Journal*, vol. 10, no. 10, pp. 8603–8617, 2023, doi: 10.1109/JIOT.2022.3220850.
- [4] P. Poomekum, A. Suriyawong, and S. Fugkeaw, "Fine-Grained and Lightweight Quantum-Resistant Access Control System With Efficient Revocation for IoT Cloud," *IEEE Open Journal of the Communications Society*, vol. 6, pp. 8652–8666, 2025, doi: 10.1109/OJ-COMS.2025.3620094.
- [5] R. P. Gupta, C. Vorakulpipat, S. Fugkeaw, and A. Witayangkurn, "A Scalable and Secure Outsourced IoT Electronic Health Records with Efficient User Revocation Using Fog-Assisted Cloud Model," in *Proc. 2024 21st International Joint Conference on Computer Science and Software Engineering (JCSSE)*, 2024, pp. 61–67, doi: 10.1109/JC-SSE61278.2024.10613723.
- [6] Z. Wang and Y. Zhou, "Analysis and Evaluation of Intel Software Guard Extension-Based Trusted Execution Environment Usage in Edge Intelligence and Internet of Things Scenarios," *Future Internet*, vol. 17, no. 1, Art. no. 32, 2025, doi: 10.3390/fi17010032.
- [7] A. Muñoz, R. Ríos, R. Román, and J. López, "A Survey on the (In)Security of Trusted Execution Environments," *Computers & Security*, vol. 129, Art. no. 103180, 2023, doi: 10.1016/j.cose.2023.103180.
- [8] A. N. Alvi, B. Ali, M. S. Saleh, M. Alkhatami, D. Alsadie, and B. Alghamdi, "Secure Computing for Fog-Enabled Industrial IoT," *Sensors*, vol. 24, no. 7, Art. no. 2098, 2024, doi: 10.3390/s24072098.
- [9] N. Sarwar, R. S. Alharthi, H. M. Mujlid, *et al.*, "Enhancing IoT Communication in Fog Computing: A Lightweight Remote User Authentication and Key Management Scheme Utilizing ECC," *Journal of Cloud Computing*, vol. 14, Art. no. 86, 2025, doi: 10.1186/s13677-025-00816-y.
- [10] W. B. Daoud, S. Othmen, M. Hamdi, *et al.*, "Fog Computing Network Security Based on Resources Management," *Journal of Wireless Communications and Networking*, vol. 2023, Art. no. 50, 2023, doi: 10.1186/s13638-023-02256-1.
- [11] H. Nguyen, S. Huda, Y. Nogami, and T. T. Nguyen, "Security in Post-Quantum Era: A Comprehensive Survey on Lattice-Based Algorithms," *IEEE Access*, vol. 13, pp. 89003–89024, 2025, doi: 10.1109/ACCESS.2025.3571307.
- [12] A. B. Shahid, K. Mansoor, Y. A. Bangash, *et al.*, "Post-Quantum Cryptographic Authentication Protocol for Industrial IoT Using Lattice-Based Cryptography," *Scientific Reports*, vol. 16, Art. no. 9582, 2026, doi: 10.1038/s41598-025-28413-8.
- [13] D. C. Lawo, R. Frantz, A. Cano Aguilera, X. Arnal I Clemente, M. P. Podleś, J. L. Imaña, I. Tafur Monroy, and J. J. Vegas Olmos, "Falcon/Kyber and Dilithium/Kyber Network Stack on Nvidia's Data Processing Unit Platform," *IEEE Access*, vol. 12, pp. 38048–38056, 2024, doi: 10.1109/ACCESS.2024.3374629.
- [14] M. A. González de la Torre, I. A. Morales Sandoval, G. T. Freitas de Abreu, and L. Hernández Encinas, "Post-Quantum Wireless-Based Key Encapsulation Mechanism via CRYSTALS-Kyber for Resource-Constrained Devices," *IEEE Access*, vol. 13, pp. 66714–66725, 2025, doi: 10.1109/ACCESS.2025.3560023.
- [15] L. Cruz-Piris, A. Marín-López, M. Álvarez-Campana, M. Sanz, J. I. Moreno, and D. Arroyo, "Measuring the Impact of Post Quantum Cryptography in Industrial IoT Scenarios," *Internet of Things*, vol. 34, Art. no. 101793, 2025, doi: 10.1016/j.iot.2025.101793.
- [16] S. Aghapour, K. Ahmadi, M. Anastasova, M. Mozaffari Kermani, and R. Azarderakhsh, "PUF-Kyber: Design of a PUF-Based Kyber Architecture Benchmarked on Diverse ARM Processors," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 12, pp. 4453–4462, 2024, doi: 10.1109/TCAD.2024.3399669.
- [17] C. Kim, S. Son, and Y. Park, "A Privacy-Preserving Authentication Scheme Using PUF and Biometrics for IoT-Enabled Smart Cities," *Electronics*, vol. 14, no. 10, Art. no. 1953, 2025, doi: 10.3390/electronics14101953.
- [18] C. Rubio García, A. Cano Aguilera, C. Stan, J. J. Vegas Olmos, S. Rommel, and I. Tafur Monroy, "Enhanced Network Security Protocols for the Quantum Era: Combining Classical and Post-Quantum Cryptography, and Quantum Key Distribution," *IEEE Journal on Selected Areas in Communications*, vol. 43, no. 8, pp. 2765–2781, 2025, doi: 10.1109/JSAC.2025.3568011.
- [19] S. Y. Moon, B. H. Jo, A. El Azzaoui, S. K. Singh, and J. H. Park, "Edge-Fog Enhanced Post-Quantum Network Security: Applications, Challenges and Solutions," *Computers, Materials & Continua*, vol. 84, no. 1, pp. 25–55, 2025, doi: 10.32604/cmc.2025.062966.
- [20] D. Aldossary, E. Aldahasi, T. Balharith, and T. Helmy, "A Systematic Literature Review on Load-Balancing Techniques in Fog Computing: Architectures, Strategies, and Emerging Trends," *Computers*, vol. 14, no. 6, Art. no. 217, 2025, doi: 10.3390/computers14060217.
- [21] M. H. Kashani and E. Mahdipour, "Load Balancing Algorithms in Fog Computing," *IEEE Transactions on Services Computing*, vol. 16, no. 2, pp. 1505–1521, 2023, doi: 10.1109/TSC.2022.3174475.
- [22] M. Ala'anzy, R. Zhanuzak, R. Akhmedov, N. Mohamed, and J. Al-Jaroodi, "Dynamic Load Balancing for Enhanced Network Performance in IoT-Enabled Smart Healthcare With Fog Computing," *IEEE Access*, vol. 12, pp. 188957–188975, 2024, doi: 10.1109/ACCESS.2024.3516362.
- [23] S. Fugkeaw, S. Rattagool, P. Jiangthiranan, and P. Pholwiset, "FPRESSO: Fast and Privacy-Preserving SSO Authentication With Dynamic Load Balancing for Multi-Cloud-Based Web Applications," *IEEE Access*, vol. 12, pp. 157888–157900, 2024, doi: 10.1109/ACCESS.2024.3485996.
- [24] X. Yu, M. Zhu, M. Zhu, *et al.*, "A Novel Location-Aware Job Scheduling Framework for Optimizing Fog-Cloud IoT Systems: Insights From Dynamic Traffic Management," *Journal of Cloud Computing*, vol. 14, Art. no. 50, 2025, doi: 10.1186/s13677-025-00772-7.
- [25] M. Vijarana, S. Gupta, A. Agrawal, M. O. Adigun, S. A. Ajagbe, and J. B. Awotunde, "Energy Efficient Load-Balancing Mechanism in Integrated IoT-Fog-Cloud Environment," *Electronics*, vol. 12, no. 11, Art. no. 2543, 2023, doi: 10.3390/electronics12112543.
- [26] S. Ijaz, S. G. Ahmad, K. Ayyub, *et al.*, "Energy-Efficient Time and Cost Constraint Scheduling Algorithm Using Improved Multi-Objective Differential Evolution in Fog Computing," *The Journal of Supercomputing*, vol. 81, Art. no. 116, 2025, doi: 10.1007/s11227-024-06550-7.
- [27] J. Wan, B. Chen, S. Wang, M. Xia, D. Li, and C. Liu, "Fog Computing for Energy-Aware Load Balancing and Scheduling in Smart Factory," *IEEE Transactions on Industrial Informatics*, vol. 14, no. 10, pp. 4548–4556, 2018, doi: 10.1109/TII.2018.2818932.
- [28] J. Singh, P. Singh, E. M. Amhoud, and M. Hedabou, "Energy-Efficient and Secure Load Balancing Technique for SDN-Enabled Fog Computing," *Sustainability*, vol. 14, no. 19, Art. no. 12951, 2022, doi: 10.3390/su141912951.

- [29] H. Wang, L. Cai, X. Hao, J. Ren, and Y. Ma, "ETS-TEE: An Energy-Efficient Task Scheduling Strategy in a Mobile Trusted Computing Environment," *Tsinghua Science and Technology*, vol. 28, no. 1, pp. 105–116, 2023, doi: 10.26599/TST.2021.9010088.
- [30] Y. Li, D. Zeng, L. Gu, A. Zhu, Q. Chen, and S. Yu, "PASTO: Enabling Secure and Efficient Task Offloading in TrustZone-Enabled Edge Clouds," *IEEE Transactions on Vehicular Technology*, vol. 72, no. 6, pp. 8234–8238, 2023, doi: 10.1109/TVT.2023.3237204.
- [31] S. Fugkeaw, R. Prasad Gupta and K. Worapaluk, "Secure and Fine-Grained Access Control With Optimized Revocation for Outsourced IoT EHRs With Adaptive Load-Sharing in Fog-Assisted Cloud Environment," in *IEEE Access*, vol. 12, pp. 82753–82768, 2024, doi: 10.1109/ACCESS.2024.3412754.
- [32] Z. Liu et al., "A Privacy-Preserving Outsourcing Computing Scheme Based on Secure Trusted Environment," in *IEEE Transactions on Cloud Computing*, vol. 11, no. 3, pp. 2325–2336, 1 July–Sept. 2023, doi: 10.1109/TCC.2022.3201401.
- [33] Y. Bai, D. He, Z. Yang, M. Luo and C. Peng, "Efficient Module-Lattice-Based Certificateless Online/Offline Signcryption Scheme for Internet of Medical Things," in *IEEE Internet of Things Journal*, vol. 12, no. 14, pp. 27350–27363, 15 July 15, 2025, doi: 10.1109/IIOT.2025.3562262.
- [34] W. Zhang, J. Wei and M. K. Khan, "PQC-Enhanced Privacy-Preserving Federated Learning for Edge-Based IoT Ecosystems," in *IEEE Transactions on Consumer Electronics*, vol. 71, no. 4, pp. 11171–11182, Nov. 2025, doi: 10.1109/TCE.2025.3601395.
- [35] A. N. Zaheer, M. Farhan, M. Rehan Naeem and M. M. Alnfai, "Quantum-Resilient Cryptographic Frameworks: Design and Analysis of Post-Quantum Algorithms for Secure and Efficient Edge-Assisted IoT Ecosystems in Consumer Electronics Devices," in *IEEE Transactions on Consumer Electronics*, vol. 71, no. 4, pp. 11258–11268, Nov. 2025, doi: 10.1109/TCE.2025.3619556.
- [36] Z. Man, Z. Yu, J. Yu, C. Gao and X. Meng, "Edge Computing in Internet of Things: Lattice-Based and Split Encryption for Post-Quantum Data Security," in *IEEE Internet of Things Journal*, vol. 12, no. 23, pp. 49327–49339, 1 Dec. 1, 2025, doi: 10.1109/IIOT.2025.3591521.
- [37] A. M. Jasim and H. Al-Raweshidy, "An Adaptive SDN-Based Load Balancing Method for Edge/Fog-Based Real-Time Healthcare Systems," in *IEEE Systems Journal*, vol. 18, no. 2, pp. 1139–1150, June 2024, doi: 10.1109/JSYST.2024.3402156.
- [38] R. M. Singh, G. Sikka and L. K. Awasthi, "LBATSM: Load Balancing Aware Task Selection and Migration Approach in Fog Computing Environment," in *IEEE Systems Journal*, vol. 18, no. 2, pp. 796–804, June 2024, doi: 10.1109/JSYST.2024.3403673.
- [39] E. Oustad, "DIST: Distributed Learning-Based Energy-Efficient and Reliable Task Scheduling and Resource Allocation in Fog Computing," in *IEEE Transactions on Services Computing*, vol. 18, no. 3, pp. 1336–1351, May–June 2025, doi: 10.1109/TSC.2025.3568255.
- [40] M. Maashi, "Elevating Survivability in Next-Gen IoT-Fog-Cloud Networks: Scheduling Optimization With the Metaheuristic Mountain Gazelle Algorithm," in *IEEE Transactions on Consumer Electronics*, vol. 70, no. 1, pp. 3802–3809, Feb. 2024, doi: 10.1109/TCE.2024.3371774.
- [41] D. Kang and H. J. Jo, "TB-Logger: Secure Vehicle Data Logging Method Using Trusted Execution Environment and Blockchain," in *IEEE Access*, vol. 11, pp. 23282–23292, 2023, doi: 10.1109/ACCESS.2023.3253626.
- [42] C. Wang, Y. Deng, Z. Ning, K. Leach, J. Li, S. Yan, Z. He, J. Cao, and F. Zhang, "Building a Lightweight Trusted Execution Environment for Arm GPUs," in *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 4, pp. 3801–3816, 2024, doi: 10.1109/TDSC.2023.3334277.
- [43] J. Zhao, H. Zhu, F. Wang, R. Lu, and H. Li, "SXGB: Secure and Efficient Vertical Federated XGBoost via Trusted Execution Environments," in *IEEE Transactions on Dependable and Secure Computing*, vol. 23, no. 2, pp. 2275–2288, Mar. 2026, doi: 10.1109/TDSC.2025.3626379.